LLM-Augmented Data Analytics

Dr Yeo Wee Kiang

NUS | Computing
National University of Singapore

The current state: How data scientists use LLMs today

- Common usage pattern:
  1. Copy data/error to ChatGPT
  2. Get suggestion
  3. Copy-paste back to code
  4. Repeat

Is this the best we can do?

I ran this and got:
TypeError: Could not convert ['ABA C'] to numeric

```
import pandas as pd

df = pd.DataFrame({
    "age": [25, 30, None, 40],
    "income": [4800, 5200, 6100, None],
    "segment": ["A", "B", "A", "C"]
})

df = df.fillna(df.mean())
print(df.head())
```

This error happens because `df.mean()` is trying to compute the mean across **all columns**, including the non-numeric column `"segment"`.

Your dataframe contains:

- `age` → numeric
- `income` → numeric
- `segment` → string (categorical)

When Pandas attempts to compute the mean of the whole dataframe, it encounters the `"segment"` column. Internally, Pandas tries to aggregate values and ends up attempting to treat strings like `"A"`, `"B"`, `"C"` as numeric, which leads to:

```sql
TypeError: Could not convert ['ABA C'] to numeric
```

Why this happens

```python
df.mean()
```

By default, Pandas tries to compute the mean column-wise. In newer versions, it may still attempt operations that involve object columns unless explicitly restricted.

Your `"segment"` column is of dtype `object`, so it cannot be averaged.

✅ Correct Solutions

Option 1: Compute mean for numeric columns only

```python
df = df.fillna(df.mean(numeric_only=True))
```

This tells Pandas:

> Only compute means for numeric columns.

This is the cleanest fix.

Ask anything

Unauthorized distribution or sharing of these materials is strictly prohibited.

Industry reality check

The Gap: Students can code, but can't architect AI-augmented systems

Skill	Traditional Curriculum	Industry Demand (2026)
Manual implementation	✓ Taught extensively	✓ Still important baseline
LLM assistance	⚠️ Allowed with disclosure / inconsistently addressed	✓ Ubiquitous (GitHub Copilot, ChatGPT)
AI system architecture	❌ Not taught	✓✓✓ Critical gap
AI governance	❌ Not taught	✓✓ Regulatory requirement

Unauthorized distribution or sharing of these materials is strictly prohibited.

What is "Deep Integration"?

Traditional (Surface-Level)

Human asks → LLM responds → Human implements

- Human is messenger between AI and code
- Linear, synchronous workflow
- No autonomous decision-making

Deep Integration

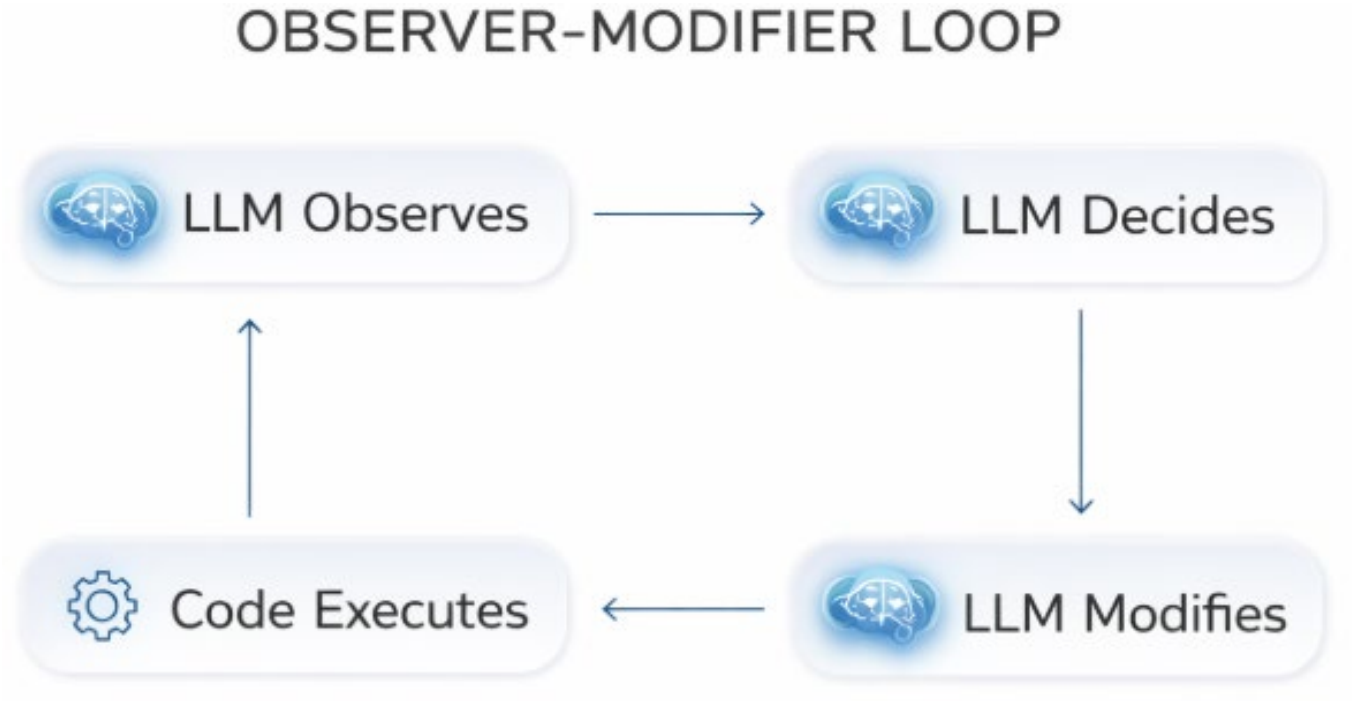
Moving from "using AI" to "**building AI collaborators**"

Code executes → LLM observes → LLM decides → LLM modifies → Loop continues

- LLM embedded IN the code
- Autonomous at runtime
- Self-adapting pipelines

Unauthorized distribution or sharing of these materials is strictly prohibited.

Observer-Modifier Loop architecture



Components:

- Execute: Run pipeline code
- Observe: Monitor outputs/metrics
- Reason: LLM analyses what happened
- Decide: LLM chooses next action
- Validate: Check decision quality
- Modify: Adapt pipeline

Unauthorized distribution or sharing of these materials is strictly prohibited.

What Deep Integration enables (impossible with chat)

Capabilities

- Self-debugging
  - Catches and fixes own errors
- Adaptive branching
  - Changes strategy based on runtime data
- Autonomous experimentation
  - Runs multiple iterations without human
- Multi-agent collaboration
  - Specialised LLMs debate decisions
- Intelligent human-in-loop
  - Knows when to ask vs. decide

These require LLM to observe, decide, and modify during execution.

Unauthorized distribution or sharing of these materials is strictly prohibited.

The Architect-Executor model

Architects don't lay bricks, but must understand structural engineering better than bricklayers.

You (Architect)	LLM (Executor)
• Defines decisions • Specifies success criteria • Designs validation • Sets ethical boundaries • Determines when human needed • Interprets domain context	• Implements iterations • Explores solution space • Executes validated strategies • Flags anomalies for review • Handles routine transforms

Unauthorized distribution or sharing of these materials is strictly prohibited.

Deep Integration patterns

Pattern	Core capability
Autonomous Feature Discovery	LLM evolves features across generations
Self-Debugging Pipelines	LLM diagnoses and fixes own errors
Real-Time Pipeline Branching	LLM makes runtime decisions altering execution path
Multi-Agent Collaboration (LangGraph)	Specialised LLMs debate and decide together
Human-in-the-Loop Orchestration	LLM decides when to interrupt human

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 1: Autonomous feature discovery

The problem	Deep Integration solution
• Manual feature engineering takes days/weeks • Limited by human creativity • Failed features forgotten (knowledge loss) • No systematic exploration	• LLM generates features across multiple generations • Tests each, keeps winners, discards losers • Learns from failures to improve next generation • Builds feature genealogy tree

- Clear criteria must be defined to determine what qualifies as a “good” feature before any generation begins.
- Validation code is written to systematically evaluate the quality and impact of each generated feature.
- The LLM improves its generation strategy over multiple iterations through meta-learning.
- Knowledge is preserved by maintaining a feature genealogy and a graveyard that capture both successful and failed features.

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 1: Autonomous feature discovery

How it works: The evolution loop

Generation N:

```
├─ LLM generates 5 feature candidates
│  (informed by: successful features, failed features, current performance)
├─ Execute each feature creation code
├─ Evaluate on validation set
├─ Keep if improves score → Feature Population
├─ Discard if doesn't → Feature Graveyard
└─ LLM meta-learning: "What worked? What failed? What to try next?"
```

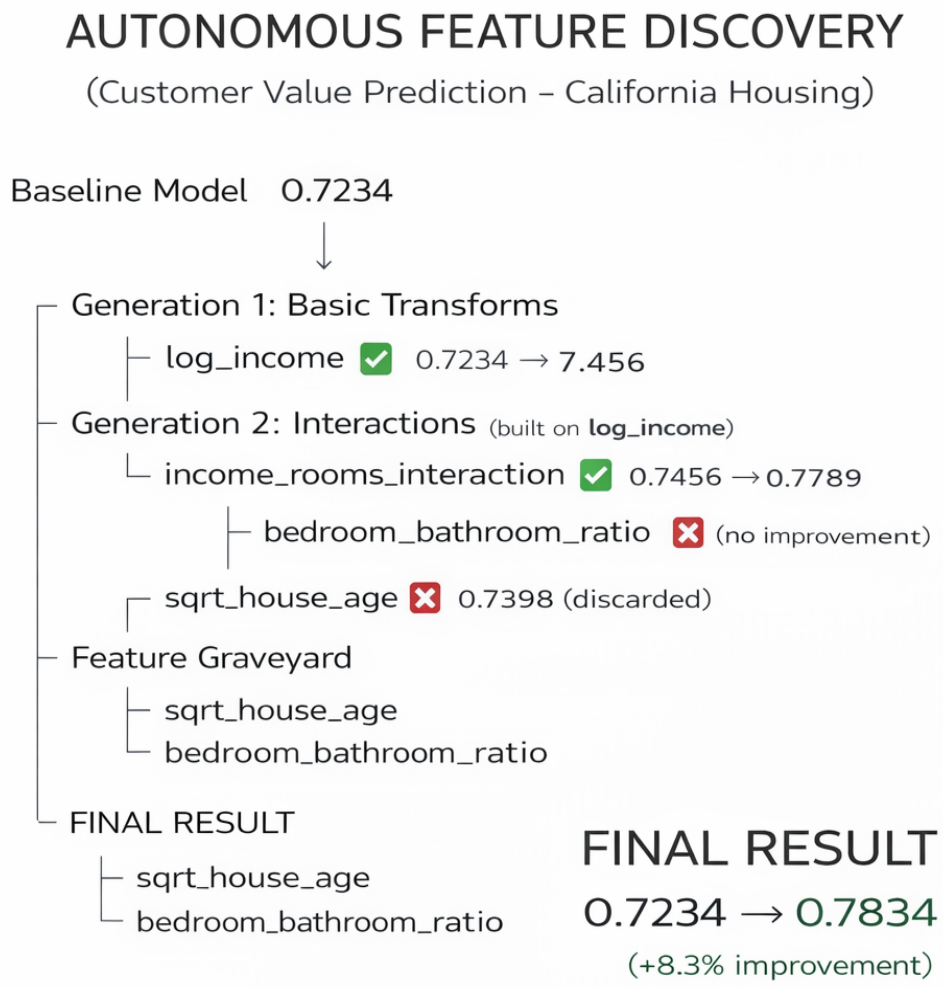
Generation N+1:

```
└─ Builds on learnings from Generation N
```


Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 1: Autonomous feature discovery

Feature Genealogy Tree visualisation



Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 1: Autonomous feature discovery

Code architecture

```
class AutoFeatureEvolution:
    def evolve_features(self, generations=3):
        baseline_score = evaluate(self.X, self.y)

        for gen in range(generations):
            # LLM generates candidates
            candidates = self._llm_generate_features(
                current_features = self.feature_population,
                failed_features = self.feature_graveyard,
                current_score = baseline_score
            )

            # You design what makes a feature "good"
            for candidate in candidates:
                new_score = self._evaluate_feature(candidate)

                if new_score > baseline_score: # you define success
                    self.feature_population.append(candidate)
                else:
                    self.feature_graveyard.append(candidate)

            # LLM learns for next generation
            self._llm_meta_learning(gen)
```

1) Calculates the baseline model score before adding any new features.
2) Runs the process for several generations (rounds of improvement).
3) The LLM generates new feature ideas, informed by successful and failed past features.
4) Each candidate feature is added and tested on validation data.
5) You define what “good” means (here: must improve the score).
6) If it improves performance, it is added to the feature population (it survives).
7) If not, it is stored in the feature graveyard (failure is remembered).
8) The LLM performs meta-learning, analysing what worked and failed to generate smarter features next round.

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 1: Autonomous feature discovery

When to use / not use

- Use when:
  - Large feature space requiring systematic exploration
  - Time-consuming manual engineering (weeks → hours)
  - Need to track what works/fails (institutional knowledge)
  - Business value from incremental improvements
- Don't use when:
  - Small datasets (<1000 samples) → overfitting risk
  - Highly regulated domains requiring full interpretability
  - Need human-explainable features for stakeholders
  - Limited API budget

Limitations:

- May generate spurious correlations
- Requires validation set for proper evaluation
- Feature complexity increases

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 2: Self-debugging pipelines

The problem	Deep Integration solution
• Errors halt pipelines requiring human diagnosis • Same errors recur in different contexts • No learning from past fixes • Generic try-except doesn't understand root cause	• LLM intercepts exceptions automatically • Diagnoses root cause (dtype vs missing vs domain violation) • Generates targeted fix (not generic fallback) • Validates fix worked, logs for learning

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 2: Self-debugging pipelines

How it works: The self-healing loop

```
Pipeline step execution:
├─ TRY: Execute code
├─ CATCH Exception:
│  ├─ LLM diagnoses root cause
│  │   • Analyses: error type, message, traceback
│  │   • Inspects: data state, previous fixes
│  │   • Returns: category (dtype | missing | domain | logic)
│  │
│  ├─ LLM generates targeted fix
│  │   • Addresses root cause, not symptom
│  │   • Adds defensive checks
│  │   • Includes explanation
│  │
│  ├─ Execute fixed code
│  │
│  └─ LLM validates output sanity
│      • Check for NaN, inf, type consistency
└─ Log fix to history (for future learning)
```

Max 3 retry attempts → If fails, escalate to human

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 2: Self-debugging pipelines

Comparison with traditional error handling

Traditional Try-Except

```python
from pyspark.sql import functions as F

try:
    # Attempt the transformation
    df = df.withColumn('new_col', F.col('col1') / F.col('col2'))
except Exception:
    # Fallback
    df = df.withColumn('new_col', F.lit(0))
```

- Same fallback for all errors
- Doesn't diagnose the root cause
- Not adaptive for different code

LLM-driven self-debugging

```python
class SparkLLMSafeProcessor:
    def safe_execute(self, code_func):
        for _ in range(3): # Retry loop
            # 1. Capture Telemetry (Metadata + Sample)
            meta = {"schema": self.df.dtypes, "preview": self.df.limit(2).collect()}

            # 2. LLM Insight & Proactive Patching
            plan = self.llm.analyze_intent(code_func.__name__, meta)
            if plan.get('action_required'):
                plan['suggested_fix'](self.context)


            try:
                # 3. Sandbox Execution & Distributed Validation
                sandbox = self.context.copy()
                code_func(sandbox)

                if self._is_valid(sandbox['df']):
                    self.context = sandbox
                    return "Success"
            except Exception as e:
                self.llm.log_failure(e, meta) # Feedback Loop


    def _is_valid(self, df):
        # Cluster-wide check for all-null or infinite values
        stats = df.select([F.count(c).alias(c) for c in df.columns]).collect()[0]
        return all(stats[c] > 0 for c in df.columns)
```


Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 2: Self-debugging pipelines

Three error types

Scenario: Corrupted loan application data

- **Error 1: Dtype Mismatch**
  - TypeError: unsupported operand type(s) for /: 'str' and 'int'
  - LLM Diagnosis: transaction_amount is string, should be numeric
  - LLM Fix: pd.to_numeric(df['transaction_amount'], errors='coerce')
  - Retry successful
- **Error 2: Missing Values**
  - TypeError: cannot multiply float by str (NaN handling)
  - LLM Diagnosis: customer_age has NaN, existing_credits has mixed types
  - LLM Fix: Impute NaN with median and convert to numeric
  - Retry successful
- **Error 3: Domain Violation**
  - ValueError: num_dependents contains negative values
  - LLM Diagnosis: Domain constraint violated (dependents can't be negative)
  - LLM Fix: df['num_dependents'].clip(lower=0)
  - Retry successful

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 2: Self-debugging pipelines

When to use / not use

- Use when:
  - Production pipelines with unpredictable data quality
  - Multiple data sources with varying formats
  - Automated ETL jobs without supervision
  - Pipeline restarts are expensive
- Don't use when:
  - Errors indicate fundamental corruption requiring human investigation
  - Highly regulated domains requiring human approval of all fixes
  - Automated fixes could mask serious systemic issues
  - Need explanation for every fix (compliance)

Limitations:
- Can mask systematic data quality problems
- Requires human review of fix_history
- Retry limits may miss valid fixes

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 3: Real-time pipeline branching

The problem	Deep Integration solution
• Hard-coded if/else rules are brittle • Can't handle novel scenarios outside predefined rules • Complex decision trees unmaintainable • No explanation of why branch was taken	• LLM analyses data characteristics at runtime • Makes branching decisions that alter execution path • Reasons about trade-offs, not just applies thresholds • Logs decision rationale for each branch

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 3: Real-time pipeline branching

How it works: Runtime decision making

Pipeline Execution:

├─ **Data Profiling**
- Calculate: imbalance ratio, dimensionality, separability
- Generate: comprehensive data profile

├─ **BRANCH POINT 1: Overall Strategy**
- LLM analyses profile
- Decides: standard | imbalance_handling | dimensionality_reduction
- Logs reasoning

├─ **BRANCH POINT 2: Imbalance Strategy (if applicable)**
- LLM evaluates severity
- Chooses: SMOTE | class_weights | balanced_ensemble
- Explains why (e.g., "SMOTE would overfit on 5% minority")

├─ **BRANCH POINT 3: Dimensionality Reduction (if applicable)**
- LLM checks samples-to-features ratio
- Decides: Apply PCA or proceed

└─ **BRANCH POINT 4: Model Selection**
- LLM chooses based on data characteristics + strategy
- Selects: LogisticRegression | RandomForest | GradientBoosting

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 3: Real-time pipeline branching

Comparison with hard-coded rules

Hard-coded approach

```
# Brittle threshold
if imbalance_ratio < 0.3:
    apply_smote()  # Always same approach
```

- Threshold (0.3) arbitrary
- Same approach (SMOTE) for all imbalanced data
- Breaks on severe imbalance
  (5% minority → overfits)

Adaptive branching

Advantages:

- Context-aware
- explains reasoning
- adapts to severity

```
# LLM reasons about data

strategy = llm_analyze_imbalance(ratio=0.05, minority_size=50)

# Returns: "balanced_ensemble" with reasoning:
# "SMOTE would create 950 synthetic from 50 real → overfitting risk.
#  BalancedRandomForest handles this natively via bootstrap."
```


Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 3: Real-time pipeline branching

Three scenarios

- **Scenario 1: Balanced Data (50:50)**
  - LLM Analysis: Balanced, high linear separability (0.987)
  - Decision: standard_classification
    - Model: LogisticRegression (simple, interpretable)
    - Reasoning: "Linear separability → simple model sufficient"
  - ROC-AUC: 0.9856
- **Scenario 2: Severely Imbalanced (5:95)**
  - LLM Analysis: Extreme imbalance (ratio 0.052), adequate samples
  - Decision: severe_imbalance_handling → balanced_ensemble
    - Model: BalancedRandomForest
    - Reasoning: "SMOTE would overfit. Native imbalance handling better."
  - ROC-AUC: 0.9345
- **Scenario 3: High-Dimensional (500 samples, 50 features)**
  - LLM Analysis: High-dim (ratio 10), overfitting risk
  - Decision: dimensionality_reduction_first → PCA
    - PCA: 50 → 23 features (95% variance)
    - Model: RandomForest
    - Reasoning: "Reduce features to prevent overfitting"
  - ROC-AUC: 0.8923

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 3: Real-time pipeline branching

Code architecture

```
class AdaptivePipelineBranching:
    def execute_adaptive_pipeline(self, data):
        # Profile data
        profile = self._profile_data(data)

        # You design: What profile characteristics matter?
        # LLM decides strategy based on profile
        strategy = self._llm_choose_strategy(profile)

        # You design: What branches are possible?
        if strategy == 'imbalance_handling':
            # Branch to imbalance handling
            approach = self._llm_choose_imbalance_strategy(profile)

            if approach == 'smote':
                data = self._apply_smote(data)
            elif approach == 'balanced_ensemble':
                # Different execution path
                return self._balanced_ensemble_branch(data)

        elif strategy == 'dimensionality_reduction':
            # Completely different branch
            return self._dimensionality_reduction_branch(data)
```


Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 3: Real-time pipeline branching

When to use / not use

- Use when:
  - Data characteristics vary widely across projects/batches
  - Need to handle diverse scenarios automatically
  - Hard-coded rules becoming complex and unmaintainable
  - Business requires adaptive strategies
- Don't use when:
  - Well-understood domain with single proven approach
  - Highly regulated requiring pre-approved methods only
  - Real-time inference where LLM latency (30-60s) prohibitive
  - Small datasets where simple approaches always work

Limitations:
- LLM decisions may not align with domain best practices
- Adds decision latency (~30-60 seconds)
- Requires validation that choices are appropriate

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 4: Multi-agent collaboration

The problem	Deep Integration solution
• Human expert committees take days (emails, meetings, scheduling) • Sequential reviews have high latency (wait for each person) • Context loss in handoffs (email threads lose nuance) • Conflicts discovered late (expensive to resolve)	• Multiple LLM agents with specialized roles • Share state via LangGraph (no manual coordination) • Debate decisions simultaneously • Coordinator resolves conflicts immediately

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 4: Multi-agent collaboration

How it works: Multi-agent architecture

Shared state eliminates manual coordination

Agent Roles

LangGraph State
(Shared state: all agents read/write)

Data Engineer Agent
- Data quality
- Preprocessing

Feature Engineer Agent
- Feature creation
- Domain knowledge

Model Architect Agent
- Model selection
- Hyperparameters

Coordinator Agent
- Synthesizes all inputs
- Resolves conflicts
- Makes final decisions

Validator Agent
- Critical review
- Challenge others

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 4: Multi-agent collaboration

LangGraph State management: Why LangGraph vs simple multiple LLM calls:

Without LangGraph (Manual):

```
response1 = llm_call("You are data engineer. Assess data...")
response2 = llm_call("You are feature engineer. Propose features...")

# Problem: response2 doesn't see response1
# You manually copy-paste context between calls
```

With LangGraph (Automatic):

```
class MLPipelineState(TypedDict):
    assessments: dict  # All agents read all assessments
    debate_log: list   # Structured debate history
    current_phase: str
    conversation_history: list

# Agents automatically access shared state
# No manual copying
```

Advantages:

- Automatic context sharing
- Structured conversation flow
- Conditional routing (validator can trigger revision)
- Complete audit trail

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 4: Multi-agent collaboration

Agent collaboration flow

Phase 1: Initial Assessment (Parallel) → **Phase 2**: Data Preparation with Validation → **Phase 3**: Model Selection Debate

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 4: Multi-agent collaboration

Agent collaboration flow

Define clear, ***distinct*** agent roles to ensure real diversity

Phase 1:
Initial Assessment (Parallel)

- Data Engineer:
  "*No missing values. Check for outliers in loan_amount. Compliance: Age-based discrimination risk*"
- Feature Engineer:
  "*Strong opportunity for debt-to-income ratios. Can create interpretable risk indicators.*"
- Model Architect:
  "*Recommend interpretable models (LogisticRegression) due to compliance requirements.*"
- Coordinator Synthesises:
  "*Build interpretable model prioritizing compliance. Use standard scaling, domain-relevant features*."

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 4: Multi-agent collaboration

Agent collaboration flow

Phase 2:
Data Preparation with Validation

- Data Engineer:
  "*1. StandardScaler for features*
  2. Check outliers in loan_amount
  3. Validate no data leakage"

- Validator:
  "*REJECTS: "Missing: What about age bias? Need fairness checks for protected characteristics.*"

- Data Engineer revises:
  "*Added: Age bias detection, disparate impact monitoring.*"

- Validator approves:
  "*APPROVED.*"

Validator can force revision via conditional edge in graph

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 4: Multi-agent collaboration

Agent collaboration flow

Every perspective considered before decision

Phase 3:
Model Selection Debate

DEBATE: Should we use LogisticRegression?

- Data Engineer:
  "*SUPPORT. Appropriate for this dataset. Features are preprocessed well.*"

- Feature Engineer:
  "*SUPPORT. LogReg will clearly show contribution of each engineered feature. Good for stakeholder communication.*"

- Validator:
  "*CONCERN. While interpretable, must ensure age doesn't become proxy for protected class. Need fairness monitoring.*"

- Coordinator:
  "*APPROVE with modifications: Use LogisticRegression + add fairness monitoring. Address validator's age bias concern through disparate impact testing.*"

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 4: Multi-agent collaboration

Code architecture

```
# Define state schema - You design what agents share
class MLPipelineState(TypedDict):
    assessments: dict
    debate_log: list
    final_decision: dict

# Build graph - You design collaboration flow
workflow = StateGraph(MLPipelineState)

# Add agents - You define agent roles
workflow.add_node("data_engineer_assess", data_engineer_assess)
workflow.add_node("validator_review", validator_review)

# Define flow - You design when validation happens
workflow.add_edge("data_engineer_assess", "validator_review")

# Conditional routing - You design approval logic
workflow.add_conditional_edges(
    "validator_review",
    should_revise,
    {"revise": "data_engineer_revise", "approve": "continue"}
)

# Compile
graph = workflow.compile()
```

1) Establish a structured, shared dictionary to track and pass data, logs, and decisions between nodes.
2) Initialise the state machine by binding it to the schema to maintain global state throughout execution.
3) Map specific Python functions to graph nodes to define the distinct roles of each agent.
4) Create direct connections between nodes to dictate the standard sequential execution of tasks.
5) Implement a routing function to determine the next node dynamically based on logical conditions or agent output.
6) Convert the graph definition into an executable program that manages state transitions and persistence.

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 4: Multi-agent collaboration

When to use / not use

- Use when:
  - Complex decisions requiring multiple expert perspectives
  - High-stakes projects where diverse views prevent errors
  - Teams with significant coordination overhead
  - Need documented, auditable decision process
  - Regulated domains requiring explanation
- Don't use when:
  - Simple tasks with established best practice
  - Time-critical decisions (adds 5-10 min latency)
  - Small projects where setup > value
  - LLM domain knowledge insufficient

Limitations:
- Higher cost (4-5x API calls vs single agent)
- Agents may agree too readily (no real conflict)
- Debates lack true domain expertise depth
- Requires prompt engineering for each agent

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 5: Human-in-the-loop orchestration

The problem	Deep Integration solution
• Fixed checkpoints interrupt unnecessarily (waste human time) • Pipeline blocks waiting for approval (hours/days delay) • No learning - same questions every run • Human sees raw outputs without analysis	• LLM decides WHEN to interrupt (uncertainty + stakes + learned preferences) • Prepares decision context (options, trade-offs, recommendation) • Timeout mechanism (proceeds if human unavailable) • Learns from decisions to reduce future interruptions

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 5: Human-in-the-loop orchestration

How it works: Intelligent Interruption

For each pipeline decision:

```
├─ LLM Assesses:
│   • Stakes (low | medium | high)
│   • Confidence (0-1)
│   • Learned preferences (has human expressed preference?)
│   • Interruption cost (human time valuable)
│
├─ Decision Rules:
│   • High stakes + Low confidence → INTERRUPT
│   • High stakes + High confidence → INTERRUPT (validation)
│   • Medium stakes + Low confidence → INTERRUPT
│   • Medium stakes + High confidence → PROCEED
│   • Low stakes (any confidence) → PROCEED
│   • Learned preference exists → FOLLOW PATTERN
│
└─ If INTERRUPT:
    ├─ Prepare decision package (options, trade-offs, recommendation)
    ├─ Request human input (30s timeout)
    ├─ If timeout: Proceed with LLM recommendation
    └─ Learn from human response for future
```


Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 5: Human-in-the-loop orchestration

Comparison with Fixed Checkpoints

Traditional fixed checkpoints

Step 1: BLOCKED "Approve preprocessing?" (always asks)
└─ Human waits 10 min → Approves StandardScaler

Step 2: BLOCKED "Approve model?" (always asks)
└─ Human waits 15 min → Approves RandomForest

Step 3: BLOCKED "Deploy?" (always asks)
└─ Human waits 10 min → Approves

Total: 35 min wait, 3 interruptions (all mandatory)

Intelligent HITL (Human-in-the-loop)

Step 1: Autonomous (low stakes, high confidence)
└─ LLM proceeds with StandardScaler (no interruption)

Step 2: Interrupt (high stakes, medium confidence)
└─ LLM presents options to human
└─ Wait up to 30 seconds for response
└─ If no response → follow LLM recommendation
└─ If response received → apply choice and store preference

Step 3: Autonomous (performance meets threshold)
└─ LLM auto-approves deployment

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 5: Human-in-the-loop orchestration

How it works: Intelligent Interruption

Run 1 (No Prior Learning)

Step 1: Preprocessing

- LLM: Confidence 0.85, Stakes low
- AUTONOMOUS: "StandardScaler is standard practice"

Step 2: Imbalance Handling

- LLM: Confidence 0.65, Stakes medium
- INTERRUPT:
  - Problem: "30% churn - SMOTE vs class_weights?"
  - Options: [SMOTE, class_weights]
  - Recommendation: class_weights (preserves real data)
  - Human: Accepts recommendation
  - Learned: "Prefer class_weights for moderate imbalance"

Step 3: Model Selection

- LLM: Confidence 0.72, Stakes high
- INTERRUPT:
  - Problem: "RandomForest vs GradientBoosting?"
  - Timeout (30s) - No human response
  - Proceeds with recommendation: RandomForest
  - Learned: "RandomForest safe default when human unavailable"

Step 4: Deployment

- LLM: Confidence 0.90, Stakes high
- AUTONOMOUS: "ROC-AUC 0.8456 ≥ 0.80 threshold → auto-approve"

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 5: Human-in-the-loop orchestration

How it works: Intelligent Interruption

Run 2 (With Learned Preferences)

Step 1: Preprocessing

- AUTONOMOUS (same as Run 1)

Step 2: Imbalance Handling

- AUTONOMOUS (NEW!)
- LLM: "Learned from Run 1: Human prefers class_weights for moderate imbalance. Proceeding autonomously."

Step 3: Model Selection

- AUTONOMOUS (NEW!)
- LLM: "Learned from Run 1: RandomForest is safe default. Similar scenario → proceeding autonomously."

Step 4: Deployment

- AUTONOMOUS (same as Run 1)

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 5: Human-in-the-loop orchestration

What human sees (Not raw outputs):

```
┌──────────────────────────┐
│  HUMAN INPUT REQUESTED   │
└──────────────────────────┘

Problem: Choose imbalance handling for 30% churn rate

LLM Analysis:
  Option 1: SMOTE (oversample minority)
    ✓ Pros: Balanced training, proven effective
    ✗ Cons: Synthetic samples, slower training

  Option 2: class_weights (weight minority higher)
    ✓ Pros: Preserves real data, faster
    ✗ Cons: Less effective for severe imbalance

Recommendation: class_weights
   Rationale: For 30% imbalance, weights sufficient
              and preserve data authenticity

Question: Given moderate imbalance, prefer
             synthetic sampling or weighting?

30 seconds to respond...
```


Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 5: Human-in-the-loop orchestration

Code architecture

```
class IntelligentHumanLoop:
    def execute_step_with_hitl(self, step_name, step_context):
        # You design: What factors determine if human needed?
        interrupt_decision = self._llm_should_interrupt(
            step_context=step_context,
            learned_preferences=self.human_preferences
        )

        if interrupt_decision['interrupt']:
            # You design: What context does human need?
            decision_package = self._llm_prepare_decision_context(
                step_context,
                interrupt_decision['reasoning']
            )

            # Request with timeout
            human_response = self._request_human_decision(
                decision_package,
                timeout=30
            )

            # You design: How to learn from response?
            self._update_human_preferences(human_response)

        else:
            # Proceed autonomously
            # Log decision for audit
```

1) Determine whether to pause the process based on task stakes, model confidence, and previous user choices.
2) Generate a comprehensive brief containing the specific data, options, and logic required for a person to make an informed choice.
3) Halt the workflow for a maximum of 30 seconds to allow for manual intervention before the system continues automatically.
4) Incorporate the user's choice into the system's memory to refine future decision-making and reduce unnecessary interruptions.

Unauthorized distribution or sharing of these materials is strictly prohibited.

Pattern 5: Human-in-the-loop orchestration

When to use / not use

- Use when:
  - Production pipelines running 24/7 without supervision
  - Human availability unpredictable
  - Cost of interruption high (breaks focus, delays)
  - Want to learn and improve autonomy over time
  - Balance needed between autonomy and oversight
- Don't use when:
  - Fully manual processes (no automation benefit)
  - 100% autonomous required (no human ever available)
  - Decisions always high-stakes (always need human)
  - LLM cannot accurately assess stakes

Limitations:
- LLM may misjudge when to interrupt
- Learned preferences may not generalize
- Timeout means some decisions without ideal input
- Initial learning period has more interruptions

Unauthorized distribution or sharing of these materials is strictly prohibited.

The paradigm shift: From tool user to System Architect

- Old mental model:
  - "How can I use LLMs to help me code faster?"
  - LLM as assistant, human as executor
- New mental model:
  - "How can I architect systems where LLMs augment human intelligence?"
  - LLM as executor, human as architect

- You are NOT learning to:
  - Outsource your thinking to LLMs
  - Become dependent on AI
- You are learning to:
  - Architect autonomous ML systems
  - Design validation frameworks
  - Govern AI decision-making
  - Know when human judgment is irreplaceable

Outcome: **Data scientists who can build AI-augmented intelligence systems**

Unauthorized distribution or sharing of these materials is strictly prohibited.